《智能体开发实战》课程实验任务书

——基于 Python 开源能力的智能体应用实践

项目模式：自然语言交互 + Python 开源项目 + Script/CLI + Skill

一、实验目的

- 结合《智能体开发实战》课程内容，通过一个小型、可运行的智能体应用项目，理解智能体如何根据用户需求进行任务理解、工具选择与任务执行。
- 掌握将成熟 Python 开源项目封装为可调用能力的基本方法，重点实践 Script/CLI 与 Skill 的轻量集成方式；在条件允许时，也可采用其他合理的工具调用方式。
- 通过 Excel、PPT、PDF、图片、音频、本地资料、文件自动化、桌面/Web 操作等典型任务，体验自然语言驱动实际软件能力的应用模式。
- 培养开源项目阅读、二次集成、接口设计、异常处理、安全控制、测试、Git 协作和项目展示能力。
- 理解智能体应用中的最小权限、用户确认、敏感信息保护、低 Token 和可追溯日志等基本工程要求。
- 强调“可运行、可交互、可解释”的小型项目实践，不要求学生从零训练大模型，也不要求重新实现成熟开源项目的底层算法。

二、实验内容

每组从下表 27 个稳定基础题目中选择 1 个。每个题目均包含一个基础任务和一个可选功能。基础任务是课程实验的主要考核内容；功能只有在基础任务稳定完成后再进行，不影响基础功能验收。推荐采用“用户自然语言 → 智能体 → Skill → Python Script/CLI → 开源项目 → 结果”的轻量路线。课程不要求必须使用 nanobot；如选择 nanobot，可将其作为智能体入口和 Skill 编排层。

编号	功能性项目题目	基础任务要求	参考开源项目	可选功能
01	AI Excel 数据分析	通过自然语言分析 Excel 数据，完成数据概览、统计、排序、趋势和异常分析，并返回可理解的分析结论。	pandas / openpyxl	增加自动生成分析图表
02	AI PPT 结构重组	读取已有 PPT，识别页面主题和章节，根据用户提出的汇报目的调整页面顺序与章节逻辑，并输出可编辑 PPT。	python-pptx	生成重组后的目录页
03	AI Office 文档审计	检查 Word/Office 文档的标题层级、空字段、缺失章节和基础格式问题，输出结构化审计结果与建议。	python-docx	增加自定义审计规则
04	AI Word 模板填充	根据 Excel/JSON 等结构化数据自动填充固定 Word 模板，批量生成文档并处理缺失字段。	python-docx	增加生成前的数据完整性预览

05	AI Word 版本比较	比较两个 Word 文档的新增、删除和修改内容，生成清晰的差异结果。	python-docx	生成修改摘要
06	AI PDF 论文解析	解析 PDF 论文的标题、章节和主要内容，支持根据自然语言问题查询论文信息。	MinerU	生成论文关键结论卡片
07	AI PDF 表格提取	从 PDF 指定页面或区域提取表格，进行基础清洗后输出 CSV/Excel 等结构化数据。	pdfplumber	增加表头识别与基础数据清洗
08	AI PDF 数据比对	从两份 PDF 中提取可比较的数据，识别新增、删除和数值变化，并给出重点变化项。	tabula-py / pandas	自动输出变化 Top 5
09	AI 扫描资料数字化	对扫描 PDF、图片等资料进行 OCR，生成可保存、可检索的数字文本，并保留基本页码对应关系。	PaddleOCR	增加关键词检索与页码定位
10	AI 文档结构导航	解析文档标题、章节和列表等结构，生成目录，并根据用户指令定位目标章节或页面。	Docling	增加跳转到对应页面/章节
11	AI PDF 页面编排	根据自然语言完成 PDF 页面提取、删除、合并、排序和旋转等操作。	pypdf	增加执行前操作预览与确认
12	AI PDF 内容标注	根据关键词或内容条件，在 PDF 中定位文字并添加高亮、下划线、批注或书签。	PyMuPDF	增加自动生成书签
13	AI 图片信息抽取	从名片、表单、截图等图片中识别文字，并按用户指定字段抽取结构化信息。	PaddleOCR	支持批量图片处理
14	AI 截图界面理解	对电脑截图进行 OCR 和区域识别，回答用户关于文字、错误提示和界面区域的问题。	Pillow + PaddleOCR / 视觉模型	增加截图区域定位
15	AI 会议语音转写与检索	将会议音频转换为带时间戳的文字，支持关键词检索、片段定位并返回相关会议内容。	FunASR	增加自动会议摘要
16	AI 本地资料检索	建立本地资料库，通过自然语言检索相关文档	Chroma	增加新文件自动导入资料库

		片段，并返回来源文件和必要上下文。		
17	AI Agent 上下文压缩	对较长的智能体对话或任务上下文进行结构化压缩，在尽量保持任务信息的情况下减少 Token 消耗。	Sentence Transformers / Python	增加压缩前后质量与 Token 对比
18	AI 文件变化监听	监听指定文件夹的新建、修改等事件，并根据预设条件触发智能体执行任务或发送通知。	watchdog	增加自然语言创建监听规则
19	AI 文件智能整理	根据文件名、类型和内容对本地文件进行分类、重命名和移动，执行前提供计划或预览。	pathlib / watchdog	增加一键撤销最近一次整理
20	AI 桌面应用控制	在受限的本地桌面环境中，通过自然语言完成打开应用、输入、点击、截图等少量预设操作。	PyAutoGUI	增加危险动作确认
21	AI 网页业务操作	针对教师提供的本地模拟业务网站，根据自然语言完成查询、填报或预约等固定业务流程。	Playwright Python	增加提交前预览与用户确认
22	AI 网页信息采集	从指定网页采集用户要求的字段，整理成 JSON/CSV/Excel 等结构化结果。	browser-use / Playwright	增加多网页结果去重
23	AI Excel 公式生成	根据自然语言需求生成 Excel 公式，写入指定单元格，并向用户解释公式作用。	openpyxl	增加公式自动验证
24	AI PPT 页面生成	根据整理好的文字内容生成固定模板风格的可编辑 PPT 页面，完成标题、正文和基础版式填充。	python-pptx	增加多种页面布局自动选择
25	AI 模型智能路由	根据任务难度、长度和类型在不同模型之间进行简单路由，并统计耗时、Token 和成本。	LiteLLM	增加 Token/成本统计展示
26	AI Agent 工具安全控制	为智能体可调用工具建立最小权限机制，包括工具白名单、目录白名单、危险操作确认等。	nanobot + Python / FastAPI（推荐）	增加完整操作审计日志
27	AI 办公任务流程编排	将已有办公 Script/Skill 按用户目标组合成简单	nanobot Skill + Python（推荐）	增加失败步骤重试或从失败步骤继续执行

		工作流，支持多个步骤之间的结果传递。		

三、实验环境

- 操作系统：Windows 10/11、Ubuntu 22.04+ 或 macOS。涉及桌面自动化时建议使用本地桌面环境；网页业务操作题目使用教师提供的本地模拟网站。
- Python：建议 Python 3.10～3.12，使用 venv、conda 或 uv 建立独立虚拟环境。
- 智能体运行环境：可使用课程讲授中介绍的智能体框架或自行选择合适框架。若使用 nanobot，可将其作为统一交互入口；课程不规定必须使用 nanobot。
- 开发工具：VS Code、PyCharm、Cursor 等；建议使用 Git 进行版本管理和两人协作。
- Python 开源依赖：根据选题安装相应库，例如 pandas、openpyxl、python-docx、python-pptx、PyMuPDF、pypdf、PaddleOCR、FunASR、watchdog、Playwright 等。
- 模型服务：可使用课程提供的模型 API、本地模型或兼容接口的模型服务。不得将真实 API Key、密码或其他敏感凭据提交到 Git 仓库。
- 数据：优先使用课程提供的公开或虚构数据，不得使用未经授权的真实敏感个人数据。
- 网络环境：涉及网页访问的项目应优先选择国内可访问网站或教师提供的本地模拟站点，不要求 Telegram、Google Docs 等海外办公服务。

四、总体要求

4.1 基本技术要求

学生应将一个成熟的 Python 开源能力转化为可被智能体调用的功能。推荐采用 Script/CLI + Skill 的方式：Python Script/CLI 负责确定性的文件处理、数据计算或外部操作；Skill 负责描述何时使用能力、参数如何组织、调用步骤和结果格式；智能体负责理解用户意图并决定是否调用该能力。

nanobot 为推荐方案而非强制方案。若使用 nanobot，可采用“nanobot + SKILL.md + Python Script/CLI”的轻量集成；不要求为了本实验额外开发 MCP Server。MCP 或其他工具调用机制可以作为技术扩展，但不能以增加框架复杂度替代核心功能。

4.2 基础任务最低完成标准

- 至少支持 3 类与题目直接相关的自然语言用户意图，并能调用对应能力完成任务。
- 至少集成 1 个真实 Python 开源项目，README 中注明项目名称、版本、许可证和实际使用方式。
- 提供清晰的 Skill 或等价工具说明，包括使用场景、参数、调用方式、结果格式和至少 2 个示例。

- 底层 Script/CLI 应尽可能能够独立运行，以便单独测试和定位问题。
- 涉及写文件、移动/删除文件、网页提交、桌面点击等有风险操作时，应设置预览或用户确认。
- 至少准备 5～10 个测试用例，覆盖正常输入、边界情况和异常情况。
- 项目应有基本运行日志；不得记录密码、API Key 等敏感信息。
- 提供 requirements.txt/pyproject.toml 和 README.md，使教师能够按照说明安装和运行。

4.3 低 Token 与安全要求

- 大文件、长文本和大量检索结果优先由 Python 程序进行筛选、统计、切分或压缩，再把必要信息交给模型。
- 工具返回值尽量采用简洁、结构化格式，避免无条件把完整文件内容放入模型上下文。
- 至少实施一项 Token、调用次数、响应时间或数据处理方面的优化，并能够说明优化思路。
- 遵循最小权限原则，限制工具可访问的文件目录和网络范围；高风险操作必须获得用户明确确认。
- 第三方开源项目须遵守对应许可证，并在项目文档中注明来源。

4.4 可选功能

每个题目只设置一个推荐功能。功能不是必做项，只有在基础功能已经稳定并通过自测后才建议实施。如果功能开发遇到困难，可以停止开发，不影响基础项目验收和基础成绩。功能的定位是“小步扩展”，不鼓励在最后阶段大幅改变系统架构或增加新的核心技术栈。

五、验收过程

阶段一：选题与方案确认（第 1～2 天）：提交选题、目标用户、3 类核心意图、总体架构、参考开源项目、Skill/Script 设计。

阶段二：核心闭环检查（第 1 周末）：至少完成“用户自然语言 → 智能体 → Skill/工具 → Python Script/CLI → 开源项目 → 结果”的一条完整链路。

阶段三：功能完善与测试（第 2 周）：完善 3 类意图、异常处理、安全确认、日志和测试用例，形成可复现的运行环境。

阶段四：基础验收（第 2 周末）：教师随机给出或从测试集抽取 2～3 条自然语言指令，学生现场完成任务。核心功能通过后即完成基础项目。

阶段五：可选功能（第 3 周，可选）：基础项目稳定后，可完成题目对应功能并进行演示。未完成功能不扣基础分。

阶段六：最终展示与答辩：每组进行总结答辩和 Demo 展示，说明需求、系统结构、开源项目集成、交互方式、安全措施和优化。

六、评分标准

评分项目	分值	评价要点
核心功能完成度	35	基础任务稳定完成；至少 3 类自然语言意图；结果正确、可复现。
智能体 + Skill/工具集成	15	自然语言到工具调用链完整；Skill/工具说明清晰；Script/CLI 可独立测试。
开源项目集成	10	正确使用 Python 开源项目，依赖和许可证说明完整，核心能力确由开源项目支撑。
用户交互体验	10	用户表达自然；必要时能够追问参数；结果清晰；错误提示友好。
安全与异常处理	10	权限边界、用户确认、敏感信息保护和异常处理合理。
测试与工程规范	10	测试用例、README、依赖管理、Git 结构、日志和代码可维护性较好。
低 Token / 性能优化	5	至少完成一项可说明、可验证的 Token、调用次数、响应时间或数据处理优化。
可选功能	5	完成题目对应功能并稳定集成；未完成不扣前述基础分。

基础项目满分 95 分；可选功能最高 5 分，总评按 100 分封顶。学生不得因未完成功能而影响基础项目成绩。

七、提交材料

- 源代码 Git 仓库或压缩包，包含核心 Script/CLI、Skill 或等价工具说明、测试代码和依赖文件。
- README.md：项目简介、用户场景、安装方法、运行方法、工具/Skill 使用方式、开源依赖及许可证、测试方法、已知问题。
- 实验报告：需求分析、总体架构、关键实现、测试结果、安全措施、低 Token/性能优化
- 演示 Demo 视频（1 分钟内）。
- 测试数据和测试用例；不得包含真实敏感个人信息。
- 如使用外部模型/API，仅提交脱敏后的配置示例，不得提交真实密钥。